Comparing Deep RL Algorithms on Game Environments

Projekt Neuroinformatik

Table of Contents

1. Introduction
2. Reinforcement Learning Methods
3. Experimental Apparatus
4. Results
5. Discussion
6. Conclusion

Project idea

- Compare two Deep RL algorithms: Rainbow DQN and PPO
- Game Environments: Flappy Bird & Super Mario Bros. (NES)
- With a Twist: Most Papers compare on equal Environment Steps, we compare on a fixed compute-time budget instead

Motivation:

Personal interest in learning Reinforcement Learning

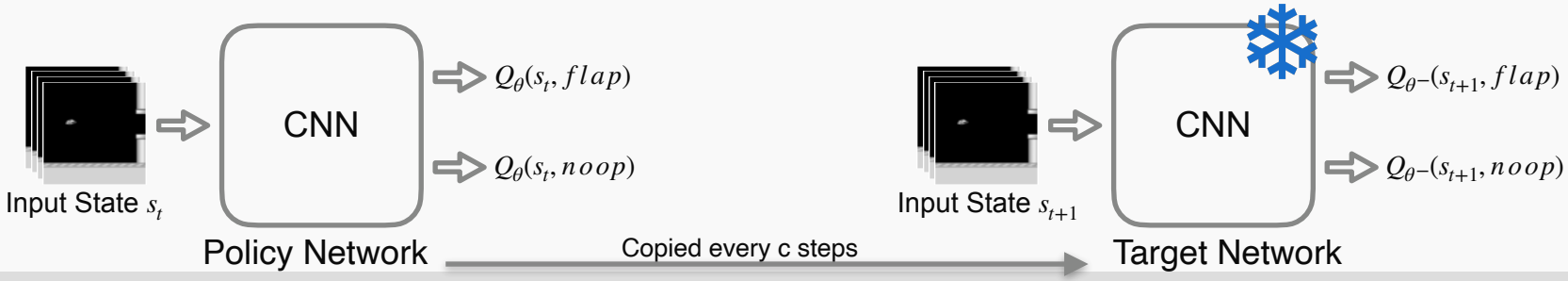
Deep-Q-Network (DQN)

Q-Value: $Q^*(s_t, a_t) = \max_{\pi} \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \;\middle|\; s_t, a_t, \pi\right]$

We learn an estimator Q_θ with the target: $y_t = r_t + \gamma \max_a Q_{\theta^-}(s_{t+1}, a)$

Transitions (s_t, a_t, r_t, s_{t+1}) are saved into a Replay Buffer $\mathcal{D}$

Loss Function: $\mathcal{L}(\theta) = \mathbb{E}_{(s_t,a_t,r_t,s_{t+1})\sim\mathcal{D}}\left[\left(y_t - Q_\theta(s_t, a_t)\right)^2\right]$



Rainbow DQN

- **Double DQN:** $y_t = r_t + \gamma\, Q_{\theta^-}\left(s_{t+1},\ \arg\max_a Q_\theta(s_{t+1}, a)\right)$
- **Dueling DQN:** $Q(s_t, a_t) = V(s_t) + A(s_t, a_t) - \frac{1}{|\mathcal{A}|}\sum_a A(s_t, a)$
- **Prioritized Experience Replay:** Transitions which are unseen, or had high Error are more likely to be selected
- **Multi-Step:** $y_t = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_a Q_{\theta^-}(s_{t+n}, a)$
- **Distributional:** Predicts a categorical distribution over N possible returns instead of their mean
- **Noisy Nets:** $y = (b + Wx) + \left(b_{\textbf{noisy}} \odot \varepsilon_b + (W_{\textbf{noisy}} \odot \varepsilon_w)x\right)$

Proximal Policy Optimization (PPO)

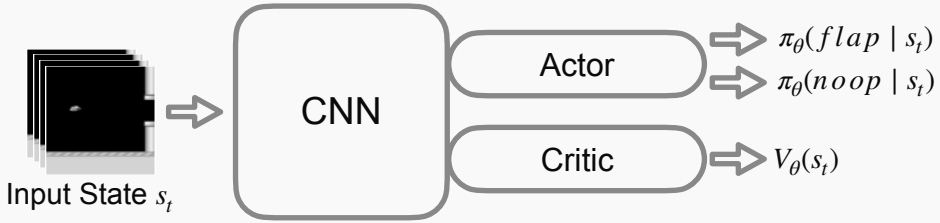
Policy Gradient Method: Estimate a Probability Distribution $\pi_\theta(a_t \mid s_t)$ over actions , improve with gradient ascent on

$\hat{g} = \hat{\mathbb{E}}_t\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\,\hat{A}_t\right] = \hat{g} = \frac{1}{N}\sum_{t=1}^{N} \ldots$, with advantage $\hat{A}_t$ estimating if an action is good or bad

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\right] \text{with } r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{old}}(a_t \mid s_t)$$

Because $\hat{A}_t = f(V_t)$ we learn to estimate $V(s_t)$: $L_t^{VF} = (V_\theta(s_t) - V_t^{targ})^2$

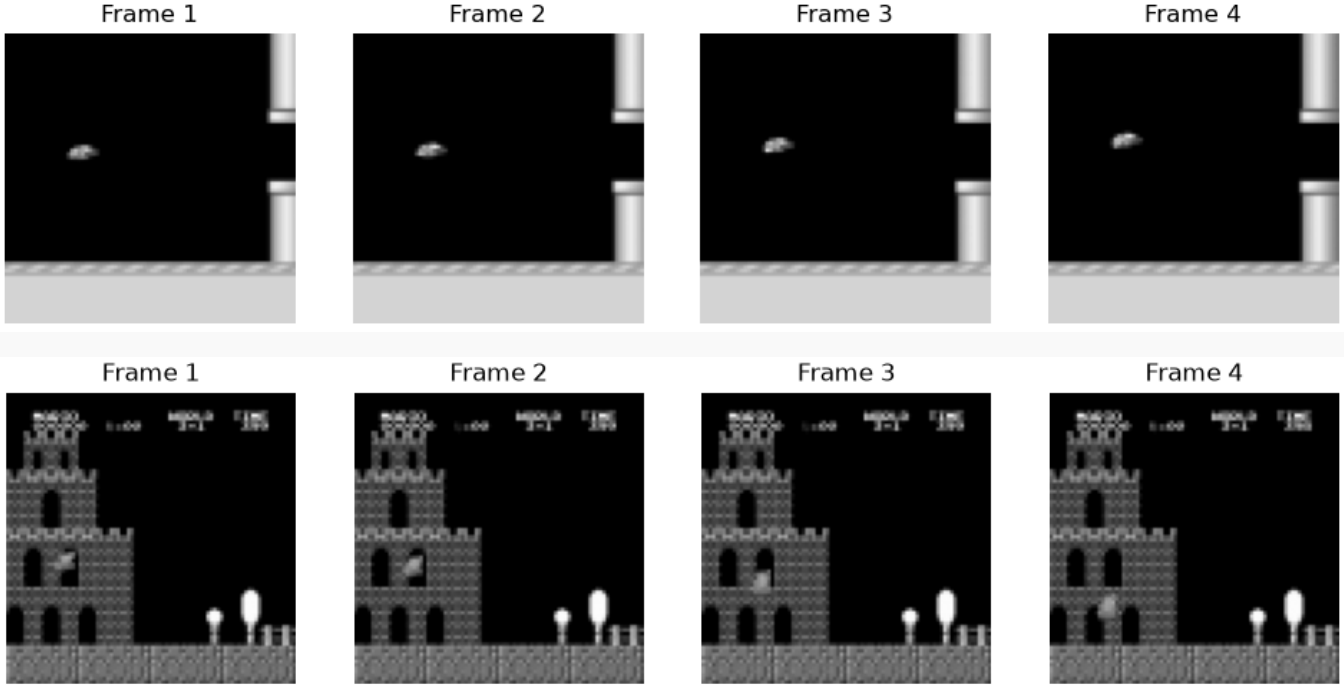
Gradient Ascent on: $L^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\right]$, with S as an entropy bonus



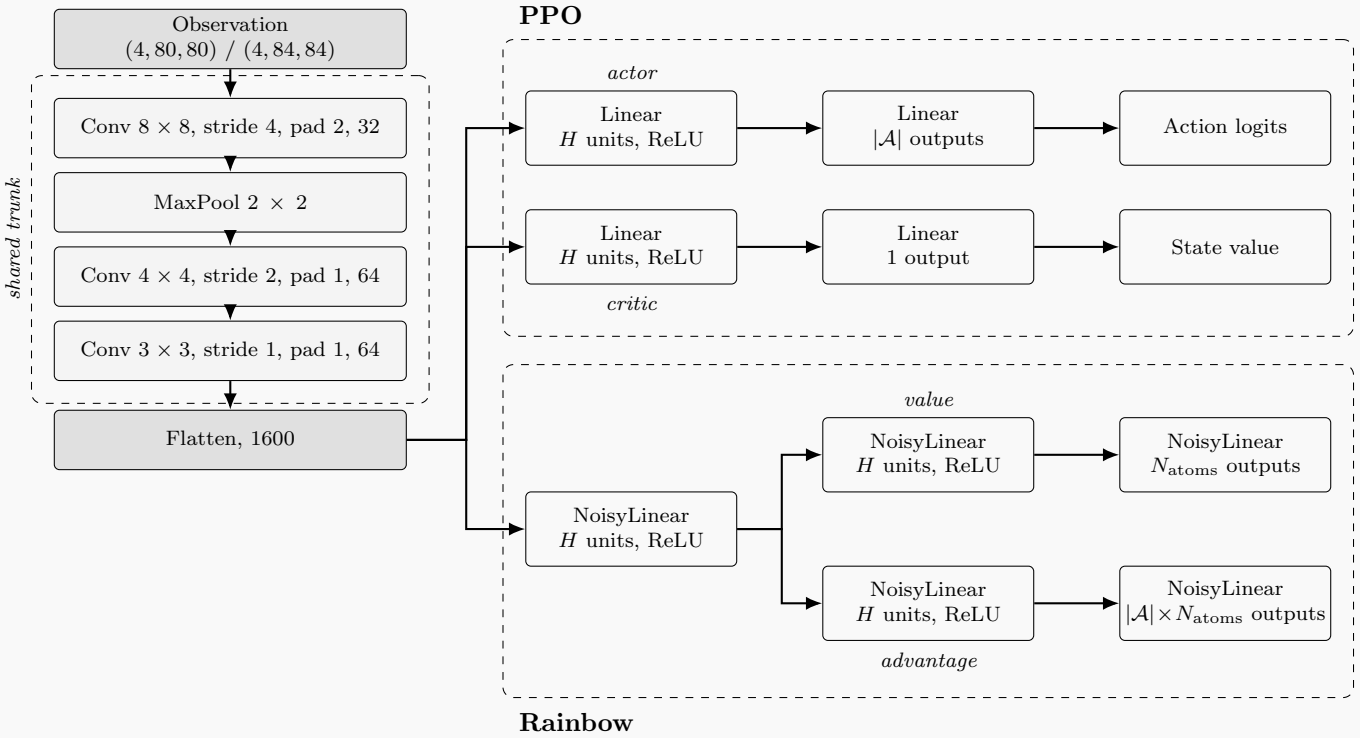
Game Environments

	Flappy Bird	**Super Mario Bros.**
Observation	$(4, 80, 80)$, grayscale	$(4, 84, 84)$, grayscale
Actions	2: flap, no-op	12: left and right, each alone or with jump, run or both; plus jump, up, down, no-op
Reward	survival: $+0.1$ per frame; pipe passed: $+1$; off screen: -0.5; collision: -1	progress: pixels moved right per frame, capped at $+5$; clock tick: -1; flag: $+50$; death: -25; minor score terms. Clipped to ± 15 per frame
Episode ends	collision only	death, flag, or the level timer running out
Stochasticity	pipe gap height (8 possibilities)	sticky actions $p = 0.25$, up to 30 no-op starts
Task metric	pipes passed	flag rate, level progress

Game Environments



CNN Architecture



Hyperparameter Optimization

- Optuna tree-structured Parzen estimator (TPE) + Hyperband pruning
- TPE: Models where good scores (e.g. mean reward) come from and proposes hyperparameter configurations to try
- Each proposal runs as a trial (shortened proxy of a whole run)
- Hyperband stops bad trials

Study	Proxy horizon	Budget	Started	Completed	Ended by
Flappy Bird PPO	1 M	200 / 24 h	200	73	trial cap
Flappy Bird Rainbow	1 M	200 / 24 h	175	55	wall clock
Mario PPO	3 M	100 / 48 h	95	49	wall clock
Mario Rainbow	3 M	100 / 48 h	13	0	failure

Table 6: PPO configurations selected by the search.

Parameter	Flappy Bird	Super Mario Bros.
Learning rate	$2.034 \cdot 10^{-4}$	$2.038 \cdot 10^{-4}$
Epochs per rollout	4	4
Minibatch size	256	64
Rollout length	4096	8192
ϵ (clip)	0.1	0.3
c_2 (entropy)	$1.514 \cdot 10^{-2}$	$1.015 \cdot 10^{-2}$
c_1 (value)	0.721	0.692
λ (GAE)	0.958	0.916
Hidden width	512	512
Resulting update rate	one per 64 steps	one per 16 steps

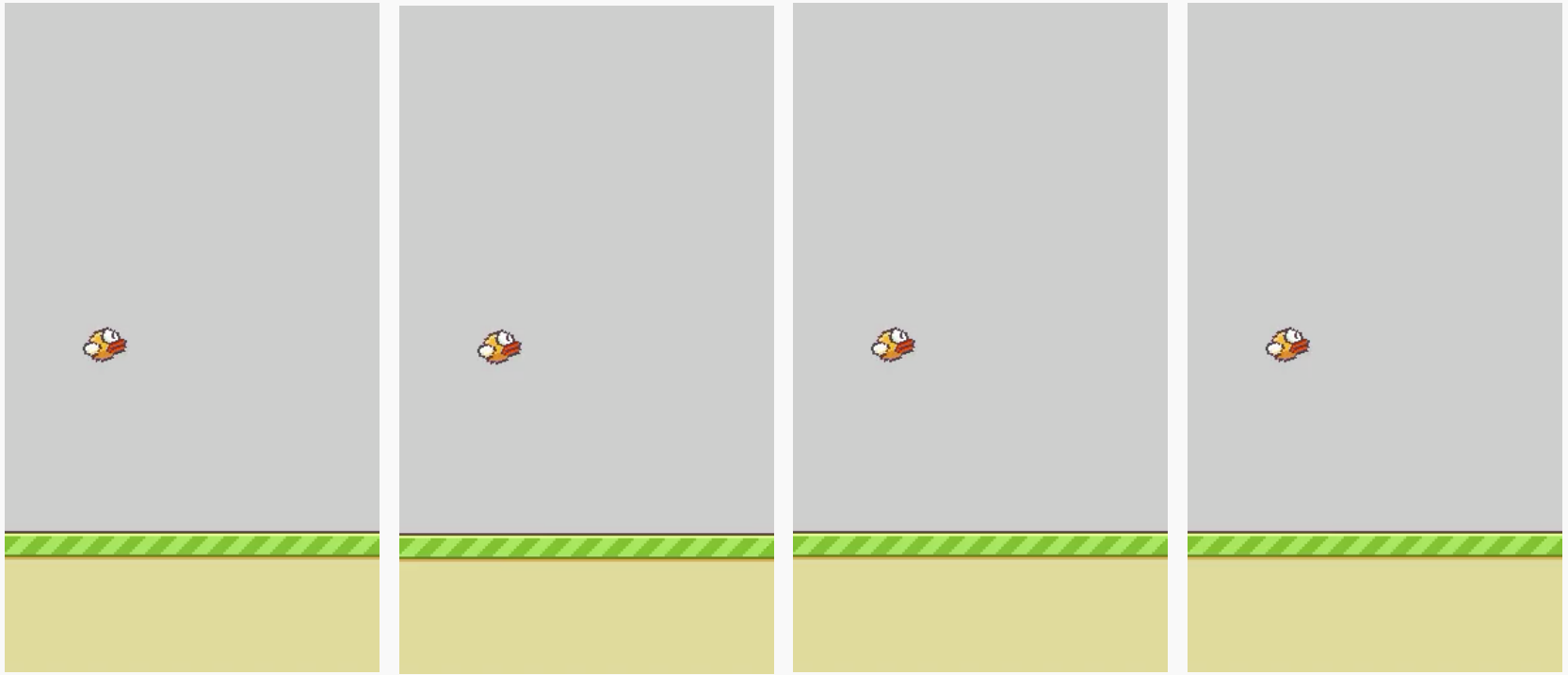
Table 7: Rainbow configurations selected by the search.

Parameter	Flappy Bird	Super Mario Bros.
Learning rate	$1.325 \cdot 10^{-4}$	$2.978 \cdot 10^{-4}$
Batch size	128	128
Target sync interval	8000	2000
n (multi-step)	3	3
α (PER)	0.620	0.625
β_0 (PER)	0.301	0.400
σ_0 (noisy)	0.425	0.559
Hidden width	256	512
Resulting update rate	one per 4 steps	one per 4 steps

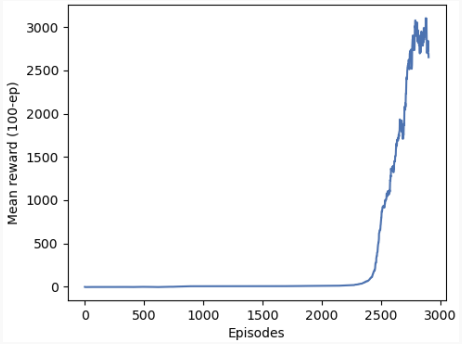
Experiment Design

- Hyperparameter Optimization → Training → Evaluation
- **Training:**
    - Instead of equal Environment steps
    - Both algorithms had the same training budget
    - Flappy Bird: 24h, Mario: 30h
    - Both ran on the same Hardware
    - Did not check different seeds

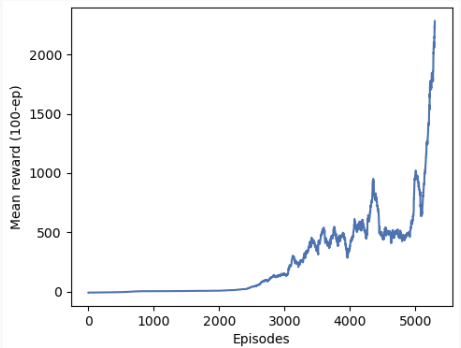
Flappy Bird Results



Flappy Bird Results



(a) Rainbow

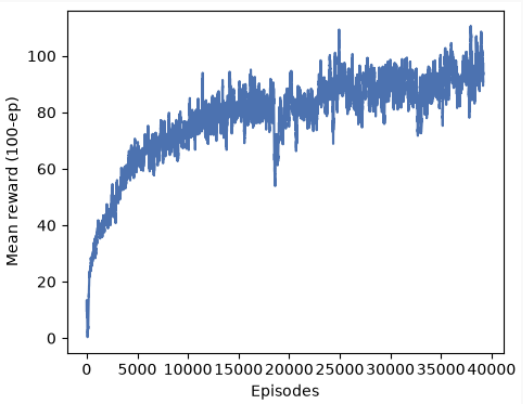


(b) PPO

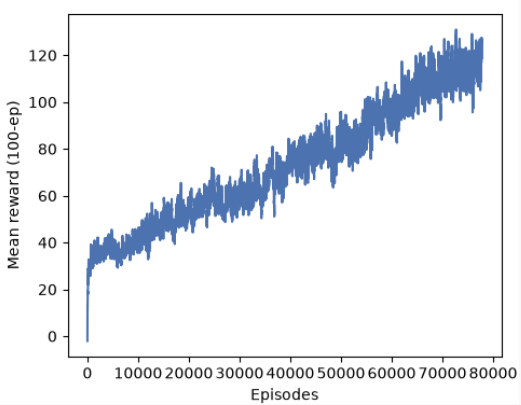
Figure 1: Reward over training on Flappy Bird, as a trailing mean over 100 episodes. Note the differing axes: Rainbow plays roughly 2 900 episodes in its 24 hours and PPO roughly 5 300.

	Rainbow	PPO
Reward	$2\,053 \pm 1\,853$	$\geq 93\,860 \pm 16\,681$
Best episode	7 237	100 000 (cap)
Pipes passed	439 ± 397	$\geq 20\,112 \pm 3\,575$
Episode length (s)	553 ± 499	$\geq 25\,253 \pm 4\,488$
Episode length (h:mm:ss)	0:09:13	$\geq$ 7:00:53
Episodes stopped by the cap	0 / 25	21 / 25

Mario Results



(a) Rainbow



(b) PPO

Figure 1: Reward over training on Super Mario Bros., as a trailing mean over 100 episodes. Each episode is played on one of the twenty training levels, drawn by a hash of its seed, so the trailing mean averages over levels of differing difficulty as well as over time. Note the differing axes: Rainbow plays roughly 39 000 episodes and PPO roughly 78 000.

Mario Results

Greedy evaluation on Super Mario Bros., 30 episodes on each of the 41 levels. Flag is the fraction reaching the flagpole and progress the fraction of the level cleared, both averaged over the levels of a block, with the deviation across levels rather than episodes.

		Rainbow		PPO	
Levels	n	Flag	Progress	Flag	Progress
Training	20	0.09 ± 0.19	0.48 ± 0.23	0.12 ± 0.28	0.58 ± 0.27
Held-out, Super Mario Bros.	9	0.00 ± 0.00	0.14 ± 0.11	0.00 ± 0.00	0.17 ± 0.14
Held-out, The Lost Levels	12	0.00 ± 0.00	0.14 ± 0.06	0.00 ± 0.00	0.08 ± 0.03
Held-out, all	21	0.00 ± 0.00	0.14 ± 0.08	0.00 ± 0.00	0.12 ± 0.10

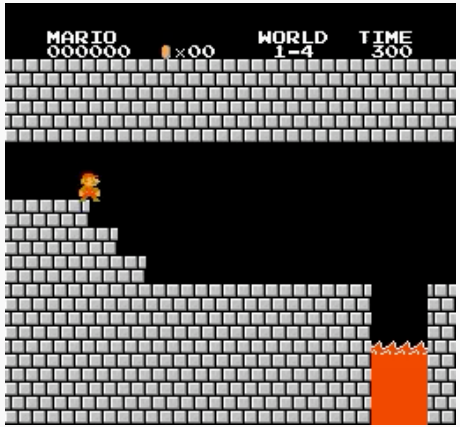
How episodes ended, as a percentage of the 600 episodes on the training levels and the 630 on the held-out ones.

	Levels	Enemy	Pit	Timeout	Flag
Rainbow	Training	51.3	33.3	6.3	9.0
	Held-out	51.3	48.7	0.0	0.0
PPO	Training	46.8	21.0	20.3	11.8
	Held-out	36.8	61.6	1.6	0.0

Mario Results







Discussion

Why did PPO outperform Rainbow on Flappy Bird to this degree?

- It’s not the learned state value V of Rainbow.
- Both critics converge exactly, to a value that can't tell the two policies apart
- We cannot state the reason, but can state 3 further experiments that could help find that out:
    - Measuring the Action-gap before deaths: margin between the two Q-values; argmax needs it to beat the estimation error.
    - Repeating the Experiment across different seeds. This could lead to different outcomes.
    - Continuing training: Rainbow could still climb.

Discussion

Why wasn’t untuned Rainbow way worse on Mario than tuned PPO?

- Either: Rainbow is less sensitive to hyperparameter variation
- Or: the failed trials happened to contain a good configuration
- Or: Mario is hard enough that tuning can't help either of them yet

Why did neither algorithm generalize?

- Maybe the algorithms haven’t learned general policies **yet**, only how to deal with known situations
- HPO wasn’t concluded with generalization in mind: no evaluation of trials on held-out levels

Conclusion

- Very fun project
- Could not say if Rainbow or PPO is better, but got interesting results
- I might run some more experiments out of personal interest and because I want the agents to beat Mario